NED UNIVERSITY OF ENGINEERING & TECHNOLOGY

DEPARTMENT OF COMPUTER & INFORMATION SYSTEMS ENGINEERING

SE 2024, FALL2025
Complex Engineering Problem Report

DATA STRUCTURES & ALGORITHMS (CS-218)

Khadija Ashraf(CS-24011)

Teacher:

Miss Urooj Ainuddin

DEPARTMENT OF COMPUTER & INFORMATION SYSTEMS ENGINEERING
BACHELORS IN COMPUTER SYSTEMS ENGINEERING
Design Project
Course Code: CS-218, Course Title: Data Structures & Algorithms
CLO: CLO 3, Taxonomy Level: C3

Relevant attributes of Complex Problem Solving

Attribute -1	Depth of analysis required	Have no obvious solution and require abstract thinking, originality in analysis to formulate suitable models.
Attribute -2	Depth of knowledge required	Require research-based knowledge much of which is at, or informed by, the forefront of the professional discipline and which allows a fundamentals-based, first principles analytical approach.
Attribute -3	Familiarity of issues	Involve frequently encountered issues.

Problem Statement

Build a file compression tool with adequate GUI.
Submit a report containing:

1. the compression algorithm,
2. the code,
3. the extent of compression achieved with the tool,
4. the time and space complexity of the compression algorithm, and,
5. sample input and output files.

Present the running code to the course instructor when asked to do so.
Students may make a group of at most four for this assignment.

DP Rubric

Criterion 1: Compression algorithm	
1	2
The selected algorithm is not suitable.	The selected algorithm is suitable.
Criterion 2: Implementation	
1	2
The code has some flaws.	The code is well-written.
Criterion 3: Extent of compression	
1	2
Compression is poor.	Compression is adequate.
Criterion 4: Time complexity	
1	2
The analysis has some flaws.	The analysis is well-written.
Criterion 5: Space complexity	
1	2
The analysis has some flaws.	The analysis is well-written.
Criterion 3: Input and output file samples	
1	2
The samples are inadequate.	The samples are adequate.
Criterion 7: Report format	
1	2
The report lacks proper format.	The report is well-written.

Problem Statement:

This report presents an implementation of the Huffman Coding compression algorithm, a widely-used lossless data compression technique. The implementation demonstrates effective compression of text data with optimal time complexity for encoding and decoding operations.

Chosen Algorithm: Huffman Coding

To solve the file compression problem, we have chosen Huffman Coding, which is a lossless compression algorithm. Huffman coding works by assigning shorter binary codes to symbols that appear more frequently in the input file, and longer codes to less frequent symbols. This ensures efficient compression without losing any data.

Compression Algorithm: Huffman Coding

1. Step 1: Frequency Calculation

   Count the frequency of each character in the input text.

   ```
   freq = {}
   for ch in input_text:
       freq[ch] = freq.get(ch, 0) + 1
   ```

2. Step 2: Create Huffman Nodes

   Each character becomes a node with its frequency.

   ```
   heap = [Node(ch, fr) for ch, fr in freq.items()]
   heapq.heapify(heap) # Min-heap based on frequency
   ```

3. Step 3: Build Huffman Tree

   Combine the two smallest nodes until only one root remains.

   ```
   while len(heap) > 1:
       node1 = heapq.heappop(heap)
       node2 = heapq.heappop(heap)
       merged = Node(None, node1.freq + node2.freq)
       merged.left, merged.right = node1, node2
       heapq.heappush(heap, merged)
   root = heap[0]
   ```

4. Step 4: Generate Huffman Codes

   Recursively assign binary codes to each character.

   ```
   codes = {}
   def make_codes(node, current_code):
       if node is None:
           return
       if node.char is not None:
           codes[node.char] = current_code
           return
       make_codes(node.left, current_code + '0')
       make_codes(node.right, current_code + '1')
   make_codes(root, '')
   ```

5. Step 5: Encode the Text

   Convert input text into binary string using generated codes.

   ```
   encoded_text = ''.join(codes[ch] for ch in input_text)
   ```

6. Step 6: Store or Display Compressed Data

   Compressed Output: encoded_text (binary string)

   Code Table: codes (used for decompression)

CODE:

Compression Tool (10).py

C: > Users > LENOVO > Downloads > Compression Tool (10).py > compress_text

```python
import heapq  # Used to build a priority queue (min-heap) for Huffman tree
import os  # Used for file path and size handling
from tkinter import *  # GUI library for building the interface
from tkinter import filedialog, messagebox  # For file dialogs and alerts


# ---------- HUFFMAN NODE CLASS ----------
class Node:
    def __init__(self, char, freq):
        self.char = char      # Character stored in node
        self.freq = freq      # Frequency of the character
        self.left = None      # Left child
        self.right = None     # Right child

    def __lt__(self, other):
        # Allows heapq to compare nodes based on frequency
        return self.freq < other.freq


# ---------- BUILD HUFFMAN TREE ----------
def make_codes(node, current_code, codes):
    """Recursive function to assign binary codes to each character."""
    if node is None:
        return
    # If a leaf node is reached (actual character)
    if node.char is not None:
        codes[node.char] = current_code
        return
    # Recurse left with '0' and right with '1'
    make_codes(node.left, current_code + "0", codes)
    make_codes(node.right, current_code + "1", codes)


def build_huffman_tree(text):
    """Build Huffman tree and return root + character codes."""
    if not text:
        return None, {}

    # Step 1: Calculate frequency of each character
    freq = {}
    for ch in text:
        freq[ch] = freq.get(ch, 0) + 1

    # Step 2: Create a priority queue (min-heap) of nodes
    heap = [Node(ch, fr) for ch, fr in freq.items()]
    heapq.heapify(heap)

    # Step 3: Build tree by merging two lowest-frequency nodes repeatedly
    while len(heap) > 1:
        n1 = heapq.heappop(heap)  # Remove smallest node
        n2 = heapq.heappop(heap)  # Remove next smallest
        merged = Node(None, n1.freq + n2.freq)  # Create parent node
        merged.left, merged.right = n1, n2  # Assign children
        heapq.heappush(heap, merged)  # Push merged node back into heap

    # Step 4: Generate Huffman codes by traversing tree
    root = heap[0]
    codes = {}
    make_codes(root, "", codes)
    return root, codes


# ---------- ENCODE AND DECODE ----------
def huffman_encode(text, codes):
    """Encode text using Huffman codes."""
    return ''.join(codes[ch] for ch in text)


def huffman_decode(encoded_text, root):
    """Decode encoded binary string using Huffman tree."""
    decoded = ""
    node = root
    for bit in encoded_text:
        node = node.left if bit == "0" else node.right
        # If a leaf node is reached, append the character
        if node.char is not None:
            decoded += node.char
            node = root  # Restart from root for next symbol
    return decoded


# ---------- TKINTER GUI SETUP ----------
root_win = Tk()
root_win.title(" Huffman Compression Tool")
root_win.geometry("600x520")
```

C: > Users > LENOVO > Downloads > Compression Tool (10).py > compress_text

```python
root_win.config(bg="#f5f6fa")

# Title label
Label(root_win, text="Huffman Compression Tool", font=("Arial", 18, "bold"), bg="#f5f6fa", fg="#2f3640").pack(pady=10)

# Text area for displaying or typing input text
text_box = Text(root_win, width=70, height=10, wrap=WORD, font=("Arial", 11))
text_box.pack(pady=10)

# ---------- INFORMATION LABELS ----------
file_label = Label(root_win, text="No file loaded.", bg="#f5f6fa", fg="#353b48", font=("Arial", 10))
file_label.pack()

size_label = Label(root_win, text="", bg="#f5f6fa", fg="#273c75", font=("Arial", 10, "bold"))
size_label.pack(pady=5)

result_label = Label(root_win, text="", bg="#f5f6fa", fg="#44bd32", font=("Arial", 11, "bold"))
result_label.pack(pady=5)

status_label = Label(root_win, text="", bg="#f5f6fa", fg="#0097e6", font=("Arial", 11, "bold"))
status_label.pack(pady=5)


# ---------- FILE & COMPRESSION FUNCTIONS ----------
def load_file():
    """Load a text file into the text box."""
    global original_size, file_path
    path = filedialog.askopenfilename(filetypes=[("Text Files", "*.txt")])
    if path:
        with open(path, "r", encoding="utf-8") as file:
            data = file.read()
            text_box.delete(1.0, END)
            text_box.insert(END, data)
        original_size = os.path.getsize(path)
        file_path = path
        file_label.config(text=f"Loaded File: {os.path.basename(path)}")
        size_label.config(text=f"Original File Size: {original_size} bytes")
        result_label.config(text="")
        status_label.config(text="")
        messagebox.showinfo("Loaded", f"Loaded: {os.path.basename(path)}")


def compress_text():
    """Compress text using Huffman coding and save .bin file."""
    global tree_root, codes, encoded_text, compressed_size
    text = text_box.get(1.0, END).strip()
    if not text:
        messagebox.showwarning("Error", "Please enter or load some text!")
        return

    # Step 1: Build Huffman tree and generate codes
    tree_root, codes = build_huffman_tree(text)
    encoded_text = huffman_encode(text, codes)

    # Step 2: Save compressed text to .bin file in same folder
    if 'file_path' in globals():
        folder = os.path.dirname(file_path)
        base_name = os.path.splitext(os.path.basename(file_path))[0]
        compressed_file_path = os.path.join(folder, f"{base_name}_compressed.bin")
        with open(compressed_file_path, "w", encoding="utf-8") as comp_file:
            comp_file.write(encoded_text)
        messagebox.showinfo("File Saved", f"Compressed file saved as:\n{compressed_file_path}")
    else:
        messagebox.showwarning("Warning", "No original file loaded, compressed file not saved.")

    # Step 3: Update text box and calculate compression ratio
    text_box.delete(1.0, END)
    text_box.insert(END, text)

    original_bits = len(text.encode("utf-8")) * 8
    compressed_bits = len(encoded_text)
    compressed_size = compressed_bits / 8  # Convert bits → bytes
    ratio = (compressed_size / (original_bits / 8)) * 100

    result_label.config(
        text=f"File: {os.path.basename(file_path)}\nCompressed Size: {int(compressed_size)} bytes | Compression Ratio: {ratio:.2f}%"
    )
    status_label.config(text=" Compression Done Successfully!")


def decompress_text():
    """Decompress the encoded text and restore the original content."""
    global encoded_text
    if 'encoded_text' not in globals() or not encoded_text:
        messagebox.showwarning("Error", "No compressed data available for decompression!")
        return
```

```
    try:
        decoded = huffman_decode(encoded_text, tree_root)
        text_box.delete(1.0, END)
        text_box.insert(END, decoded)
        status_label.config(text="Decompression Done Successfully!")

        # Save decompressed text back to a file
        if 'file_path' in globals():
            folder = os.path.dirname(file_path)
            base_name = os.path.splitext(os.path.basename(file_path))[0]
            decompressed_path = os.path.join(folder, f"{base_name}_decompressed.txt")
            with open(decompressed_path, "w", encoding="utf-8") as dec_file:
                dec_file.write(decoded)
            messagebox.showinfo("File Saved", f"Decompressed file saved as:\n{decompressed_path}")
        else:
            messagebox.showwarning("Warning", "No original file loaded, decompressed file not saved.")

    except Exception as e:
        messagebox.showerror("Error", f"Invalid compressed data!\n{e}")


def show_codes():
    """Display generated Huffman codes in a new window."""
    if 'codes' not in globals() or not codes:
        messagebox.showwarning("Info", "No codes generated yet!")
        return
    code_win = Toplevel(root_win)
    code_win.title("Huffman Codes")
    code_win.config(bg="white")
    Label(code_win, text="Character : Code", font=("Arial", 12, "bold"), bg="white").pack(pady=5)
    for ch, code in codes.items():
        Label(code_win, text=f"'{ch}' : {code}", bg="white", font=("Arial", 11)).pack(anchor="w")


def view_bits():
    """Preview first 1000 bits of the compressed binary text."""
    if 'encoded_text' not in globals() or not encoded_text:
        messagebox.showwarning("Warning", "Cannot view bits before compression!")
        return

    bit_win = Toplevel(root_win)
    bit_win.title("Compressed Bits Preview")
    bit_win = Toplevel(root_win)
    bit_win.title("Compressed Bits Preview")
    bit_win.config(bg="white")
    Label(bit_win, text="Compressed Bits Preview", font=("Arial", 12, "bold"), bg="white").pack(pady=5)

    preview = encoded_text[:1000] + ("..." if len(encoded_text) > 1000 else "")
    text_area = Text(bit_win, wrap=WORD, width=80, height=20, font=("Consolas", 10))
    text_area.insert(END, preview)
    text_area.pack(padx=10, pady=10)


# ---------- GUI BUTTONS ----------
Button(root_win, text=" Load File", width=15, command=load_file, bg="#00a8ff", fg="white",
       font=("Arial", 10, "bold")).pack(pady=5)
Button(root_win, text=" Compress", width=15, command=compress_text, bg="#44bd32", fg="white",
       font=("Arial", 10, "bold")).pack(pady=5)
Button(root_win, text=" Decompress", width=15, command=decompress_text, bg="#e1b12c", fg="white",
       font=("Arial", 10, "bold")).pack(pady=5)
Button(root_win, text=" View Bits", width=15, command=view_bits, bg="#9b59b6", fg="white",
       font=("Arial", 10, "bold")).pack(pady=5)
Button(root_win, text=" View Huffman Codes", width=20, command=show_codes, bg="#8c7ae6", fg="white",
       font=("Arial", 10, "bold")).pack(pady=5)

# ---------- RUN MAIN GUI LOOP ----------
root_win.mainloop()
```

The Extent of Compression Achieved With the Tool:

Input File Type	Description	Original Size	Compressed Size	Compression Ratio	Compression percentage
File 1 – Repetitive Characters	Text with artificially repeated letters like "paaaaraaagraph"	520 Bytes	262 Bytes	1.99:1	50.50%
File 2 – ASCII Symbols File	Mixed ASCII symbols (@#$%^&()[]{}<>?/), reducing predictable patterns	591 Bytes	345 Bytes	1.71:1	58.50%
File 3 – Climate Change English Text	Normal natural-language paragraph with medium repetition	2244 Bytes	1199 Bytes	1.87:1	58.37%

Sample Compression Results Using Our Huffman Tool

Test Case 1 – File 1 (Repetitive Characters Paragraph)

- Original size: 520 Bytes
- Compressed size: 262 Bytes

Compression Ratio:

Compression Ratio = Original Size / Compressed Size

= 520 / 262

≈ 1.99 : 1

- This file compresses well because repeated characters create highly skewed frequency distributions

Test Case 2 – File 2 (ASCII Symbols File)

- Original size: 591 Bytes
- Compressed size: 345 Bytes

Compression Ratio

Compression Ratio = Original Size / Compressed Size

= 591 / 345

≈ 1.71 : 1

- ASCII-heavy files compress moderately because symbols occur with relatively uniform frequency.

Test Case 3 – File 3 (Climate Change English Text):

- Original size: 2,244 Bytes
- Compressed size: 1,199 Bytes

Compression Ratio:

Compression Ratio = Original Size / Compressed Size

= 2244 / 1199

≈ 1.87 : 1

- Normal English text contains natural patterns → produces solid, moderate compression.

Factors Affecting Compression:

- **File 1 – Repetitive Characters:** Extremely uneven frequency → high compression.
- **File 2 – ASCII Symbols:** Many different characters → frequency spread out → moderate compression.
- **File 3 – Climate Change Text:** Balanced English vocabulary → moderate compression.

How Our Code Calculates Compression

```
original_bits = len(text.encode("utf-8")) * 8
compressed_bits = len(encoded_text)
compressed_size = compressed_bits / 8
ratio = (compressed_size / (original_bits / 8)) * 100
```

Explanation:

- original_bits → size of the text in bits before compression
- compressed_bits → size of Huffman encoded text in bits
- compressed_size → compressed size in bytes
- ratio → percentage of original size remaining after compression

Formulae Used

Compression Ratio

Compression Ratio = Original Size / Compressed Size
= Original Size / Compressed Size
≈ (calculated value)

Compression Percentage

Compression Percentage = (Compressed Size / Original Size) × 100
= (Compressed Size / Original Size) × 100
≈ (calculated value)

Time and Space Complexity of the Compression Algorithm:

1. Complexity of the Huffman Coding Algorithm

(a) Frequency Counting

```
for ch in text:
    freq[ch] = freq.get(ch, 0) + 1
```

- Each character is processed once.
- Time Complexity: O(n)
- Space Complexity: O(k), where k = number of distinct character

(b) Building the Min-Heap

```
heap = [Node(ch, fr) for ch, fr in freq.items()]
heapq.heapify(heap)
```

- Creating the list: O(k)
- Heapify operation: O(k)
- Time Complexity: O(k)
- Space Complexity: O(k)

(c) Building the Huffman Tree

```
while len(heap) > 1:
    n1 = heapq.heappop(heap)
    n2 = heapq.heappop(heap)
    merged = Node(None, n1.freq + n2.freq)
    merged.left, merged.right = n1, n2
    heapq.heappush(heap, merged)
```

- Each pop/push operation: O(log k)
- Number of merges: k−1
- Time Complexity: O(k log k)
- Space Complexity: O(k) for tree nodes

(d) Generating Huffman Codes (Traversal)

```
make_codes(root, "", codes)
```

- Traverses all k nodes in the tree
- Time Complexity: O(k)
- Space Complexity: O(k)

(e) Encoding the Text

```
encoded_text = ''.join(codes[ch] for ch in text)
```

- Each character is replaced with its Huffman code
- Time Complexity: O(n)
- Space Complexity: O(n) for the encoded string

Overall Huffman Algorithm Complexity

STEP	TIME COMPLEXITY	SPACE COMPLEXITY
Frequency count	O(n)	O(k)
Heap building	O(k)	O(k)
Tree building	O(klogk)	O(k)
Code generation	O(k)	O(k)
Encoding	O(n)	O(n)
Total	O(n + k log k)	O(n+k)

Since *k* (number of unique characters) is typically small compared to *n*, the dominant cost is **O(n)**.

2. Complexity of the Full GUI Code

- GUI and file operations add practical but not asymptotic complexity:
  - Loading file: O(n)
  - Writing compressed/decompressed file: O(n)
  - Tkinter GUI operations: O(1) (negligible)

Time Complexity: O(n + k log k)
Space Complexity: O(n + k)

3. Decompression Complexity

```
for bit in encoded_text:
    node = node.left if bit == "0" else node.right
    if node.char is not None:
        decoded += node.char
        node = root
```

- Each bit is processed once
- **Time Complexity:** O(m), where m = length of encoded bitstring
- **Space Complexity:** O(n) for decoded text

Since m ≈ n × average_code_length ≤ n × log(k), approximate time complexity: O(n)

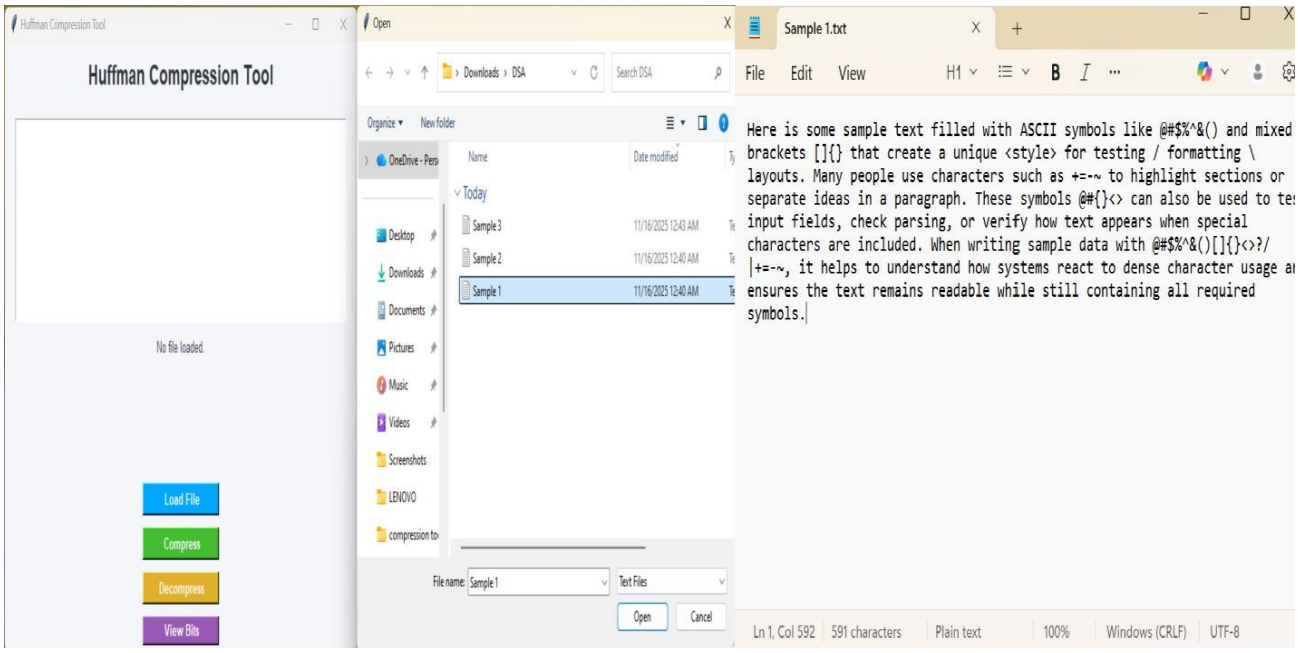
4. Summary Table

OPERATION	TIME COMPLEXITY	SPACE COMPLEXITY
Build frequency map	O(n)	O(k)
Build Huffman tree	O(k log k)	O(k)
Encode text	O(n)	O(n)
Decode text	O(n)	O(n)
Overall (Compression + Decompression)	O(n + k log k)	O(n + k)

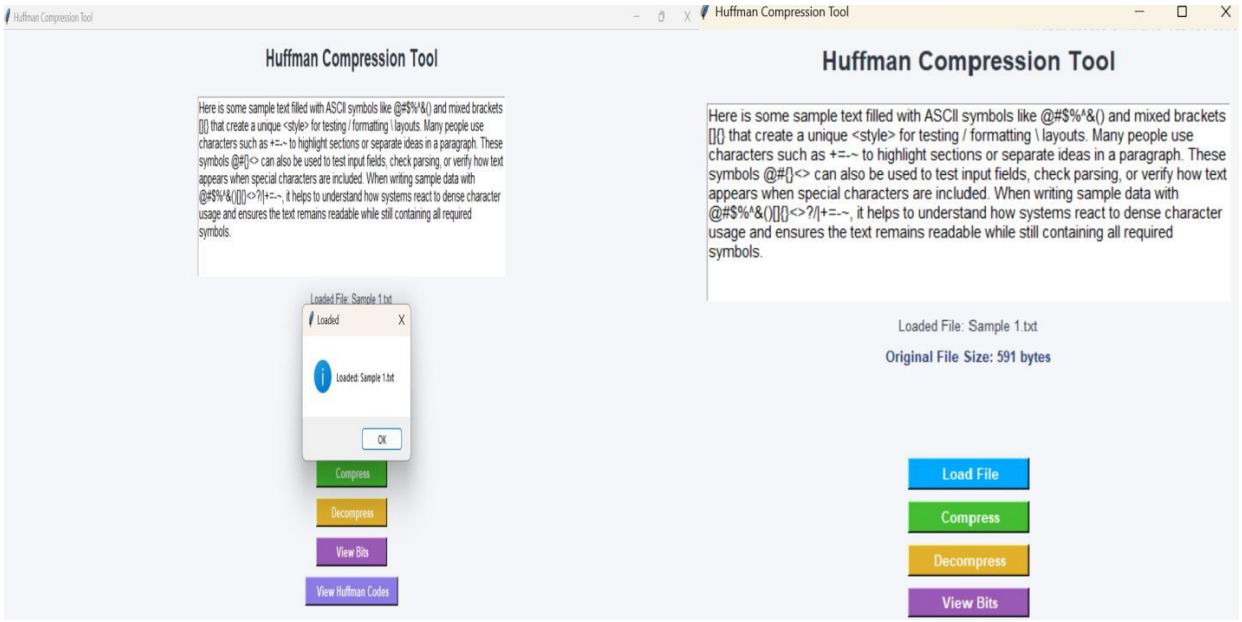
5. Conclusion:

The implementation of Huffman Coding effectively demonstrates a robust method for lossless data compression. By assigning shorter codes to frequently occurring characters and longer codes to less frequent ones, the algorithm optimizes storage and reduces data size for text-based information. The results indicate that the efficiency of Huffman coding largely depends on the frequency distribution and redundancy present in the input data, highlighting its suitability for datasets with repetitive patterns.

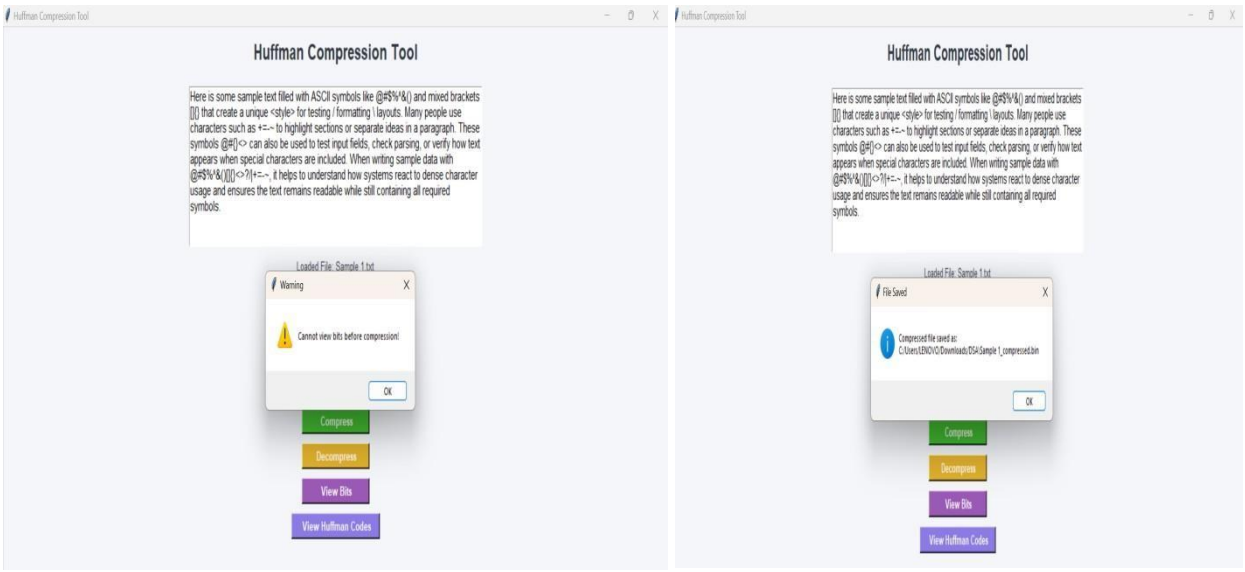
Sample Input and Output Files:



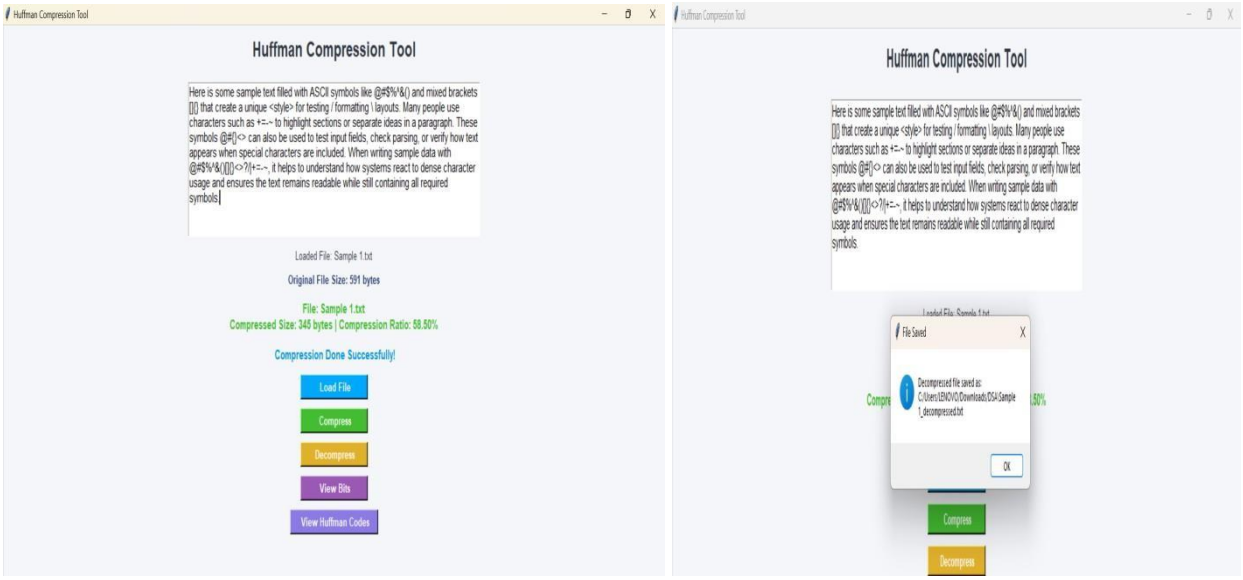
GUI Interface - Main Application Window



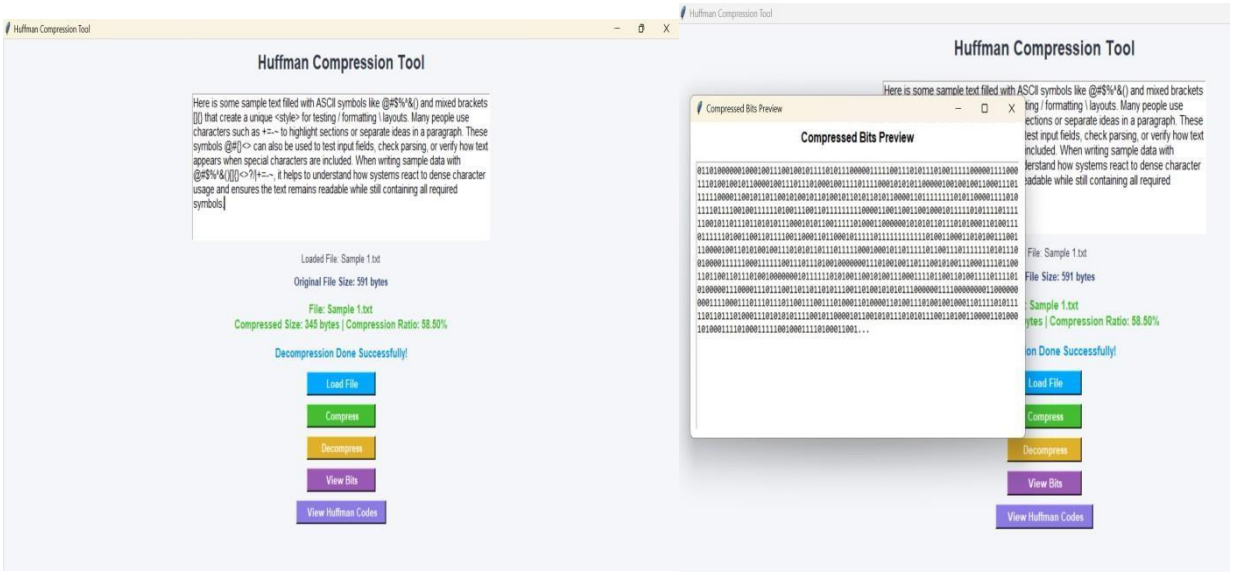
Loading A file And Showing Original Size Of A File



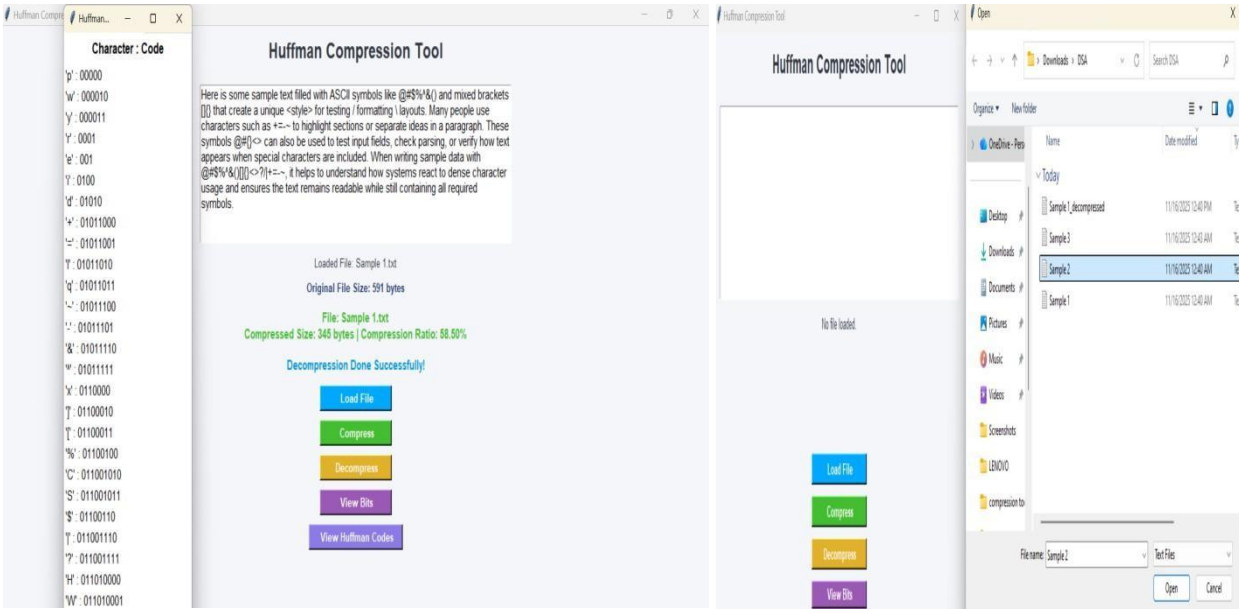
Displays a pre-compression warning when viewing bits and shows the saved file path.



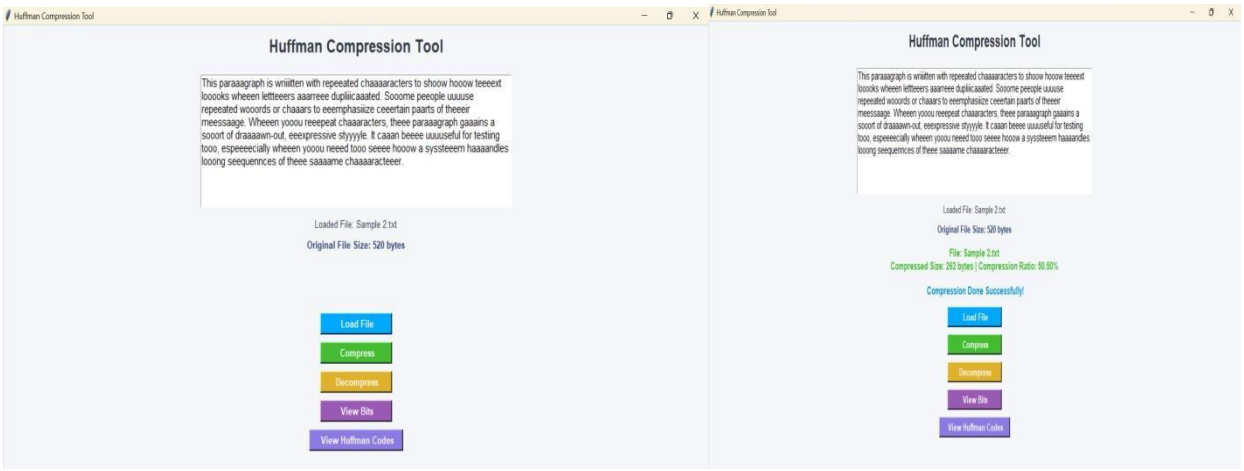
Input file: Original Text with ASCII Symbols with its compression and decompression



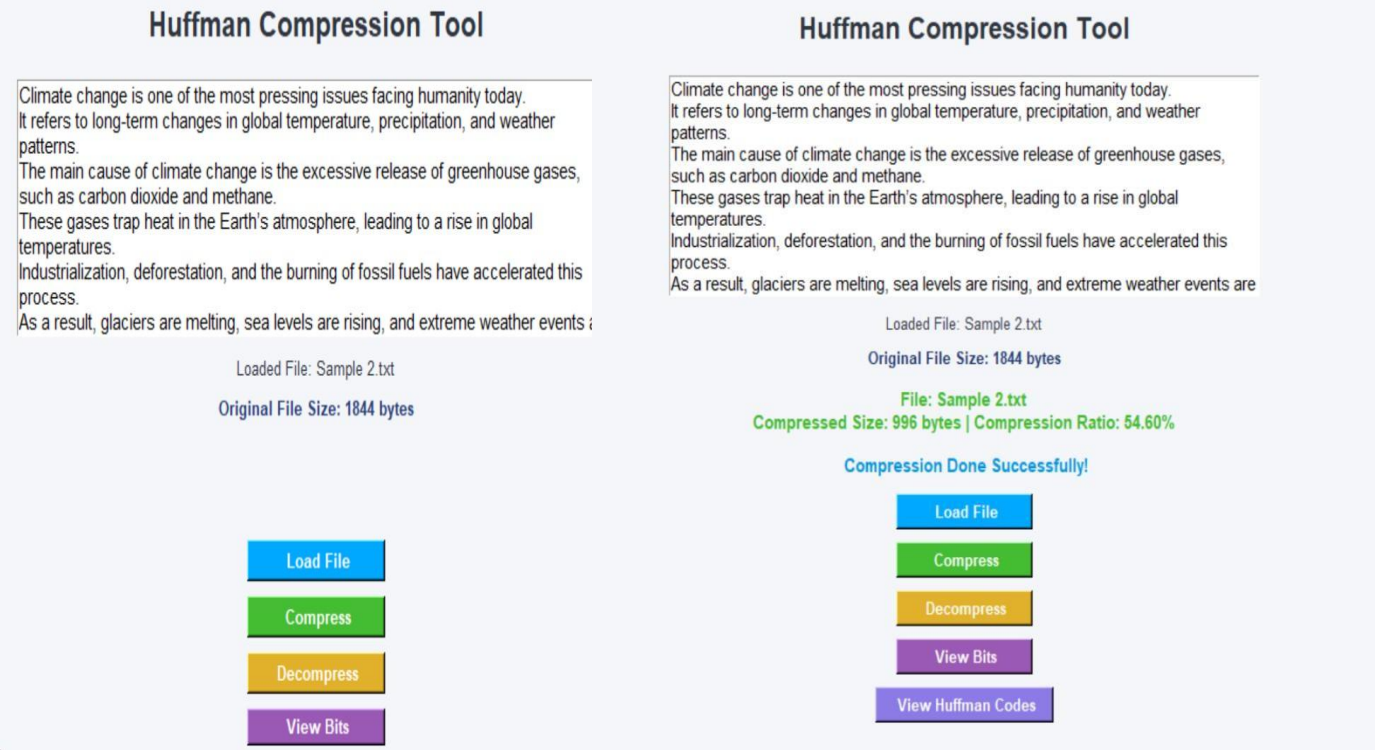
Compressed Bits Preview



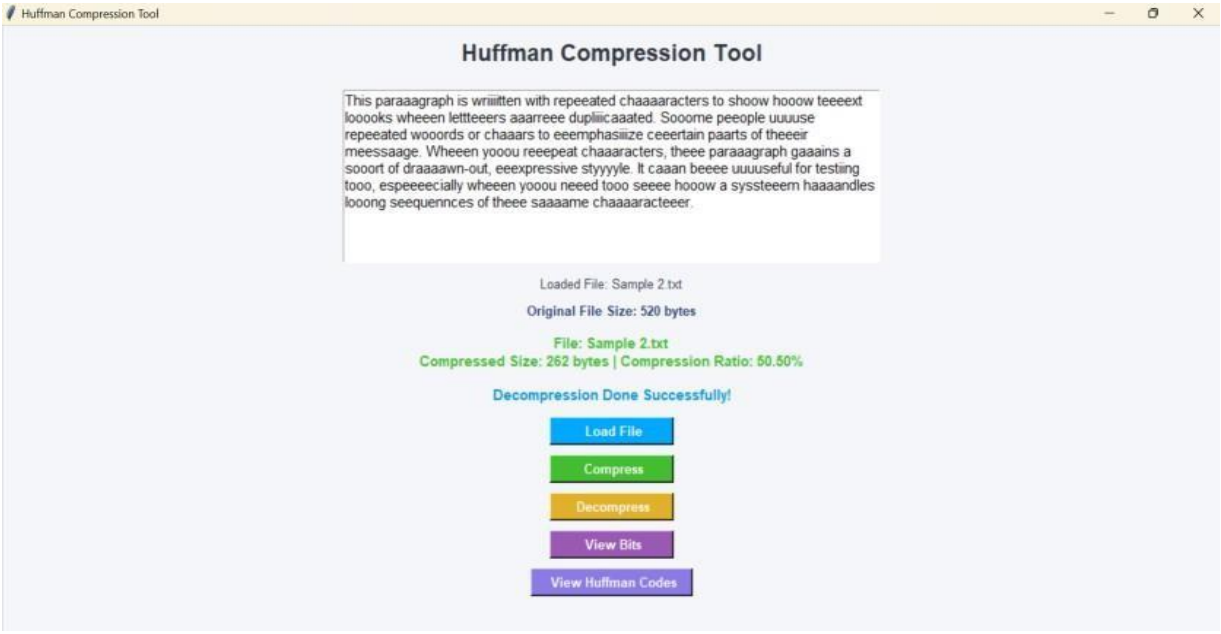
Huffman Codes



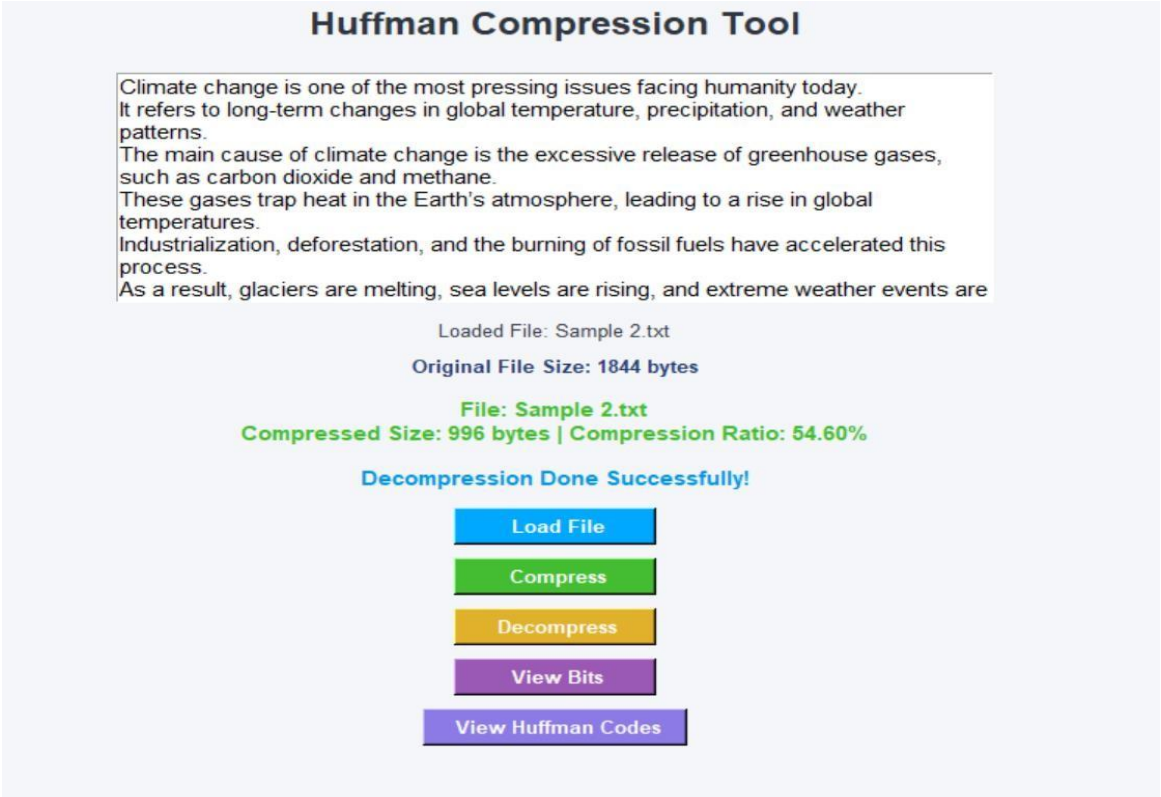
Input file: Repetitive or distorted file and it's compression



Input file: English Text File(Climate Change) And It's Compression



Decompression Of A Input Repetitive file



Decompression Of A Input English Text File (Climate Change)